

GDC-Schema → RDF/OWL

Semantic ETL des Wissensnetzes: vom flachen Dictionary zum semantischen Graph

Projekt DataBridge · 26ss_CB_DataBridge · Teilbereich Wissensnetz (Marcel Thiel)

Status: Entwurf v0.1 · Stand: 20.08.2026

Dieses Dokument konkretisiert den ETL-Baustein aus dem Wissensnetz-Konzept: Wie werden die flachen GDC-Metadaten regelbasiert in ein RDF/OWL-Wissensnetz überführt? Es beschreibt den ETL-Ablauf, die Mapping-Regeln (Konstrukt für Konstrukt), ein durchgerechnetes Beispiel und die noch offenen Designentscheidungen.

1 Ausgangslage: das GDC-Datenmodell

Die Genomic Data Commons beschreibt ihr Datenmodell nicht in OWL, sondern als **Sammlung flacher YAML-/JSON-Schema-Dictionaries**. Jede Entität („node“, z. B. case, sample, diagnosis) ist eine YAML-Datei mit **properties** (Feldern mit Typ und teils enum-Wertelisten), **links** (typisierte Beziehungen zu anderen Nodes) und einer required-Liste. Zusammen bilden die Nodes und Links einen gerichteten Graphen – aber ohne formale Semantik, ohne Reasoning und ohne Anbindung an biomedizinische Ontologien.

Genau diese Lücke schließt das Mapping: Aus flachen Feldern werden Klassen, Properties und Axiome, und aus enum-Werten werden – wo möglich – Verweise auf etablierte Ontologien (NCIt, Disease Ontology, ICD-O).

2 Der ETL-Ablauf im Überblick

Der Mapping-Schritt ist die dritte Stufe eines vierstufigen ETL-Prozesses. Er erzeugt zwei getrennte Ausgaben: das Schema (TBox) und die Instanzdaten (ABox), die beide in den Triple-Store geladen werden.

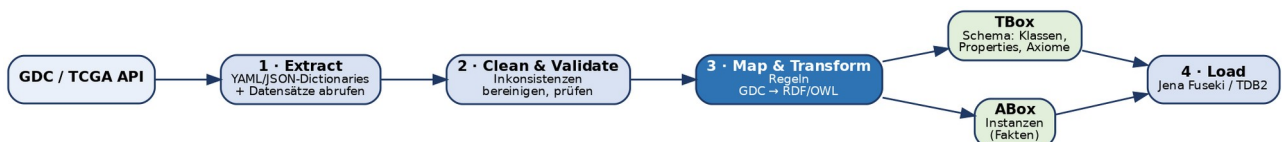


Abb. 1: Der Semantic-ETL-Ablauf. Der Map-&-Transform-Schritt trennt Schema (TBox) und Fakten (ABox); beide werden in Jena Fuseki/TDB2 geladen.

3 Die Mapping-Regeln

Das Mapping ist regelbasiert: Jedes GDC-Konstrukt hat ein festes Gegenstück in RDF/OWL. Die folgende Grafik zeigt die Zuordnung, die Tabelle darunter ergänzt je ein Beispiel.

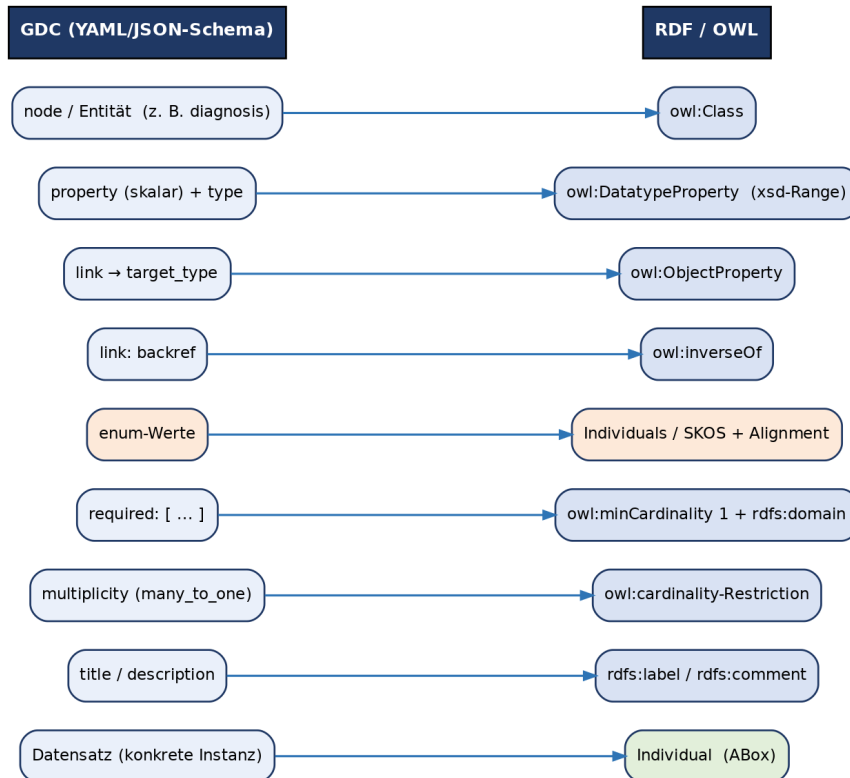


Abb. 2: Konstrukt-für-Konstrukt-Zuordnung von GDC (YAML/JSON-Schema) nach RDF/OWL. Orange hervorgehoben: enum-Werte, die auf externe Ontologien abgebildet werden.

GDC-Konstrukt	RDF/OWL-Gegenstück	Beispiel
node / Entität	owl:Class	db:Diagnosis
property + type	owl:DatatypeProperty (xsd-Range)	db:ageAtDiagnosis
link → target_type	owl:ObjectProperty	db:describesCase
backref	owl:inverseOf	db:hasDiagnosis
enum-Werte	Individuals / SKOS + Alignment	→ ncit:*
required: [...]	owl:minCardinality 1 + rdfs:domain	Pflichtfeld
multiplicity	owl:cardinality-Restriction	many_to_one
title / description	rdfs:label / rdfs:comment	Beschriftung
Datensatz (Instanz)	Individual (ABox)	db:diag-TCGA-A1

4 Konkretes Beispiel: der diagnosis-Node

Ausgangspunkt ist ein (vereinfachter) Auszug des GDC-diagnosis-Nodes:

```

id: diagnosis
title: Diagnosis
category: clinical
links:
  - name: cases
    backref: diagnoses
    target_type: case
    multiplicity: many_to_one
    required: true
required:
  - primary_diagnosis
properties:
  primary_diagnosis:
    description: Text term for histologic diagnosis
  enum:
    - "Acinar cell carcinoma"

```

```
- "Adenocarcinoma, NOS"
age_at_diagnosis:
  type: integer
  minimum: 0
```

Daraus entsteht das **Schema (TBox)**:

```
@prefix db: <http://databridge.hka.onto#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ncit: <http://purl.obolibrary.org/obo/NCIT_> .

db:Diagnosis a owl:Class ;
  rdfs:label "Diagnosis" ;
  rdfs:comment "Text term for histologic diagnosis" .

db:describesCase a owl:ObjectProperty ;      # link: cases
  rdfs:domain db:Diagnosis ; rdfs:range db:Case ;
  owl:inverseOf db:hasDiagnosis .           # backref

db:primaryDiagnosis a owl:ObjectProperty ;   # enum -> Alignment
  rdfs:domain db:Diagnosis ; rdfs:range ncit:Neoplasm .

db:ageAtDiagnosis a owl:DatatypeProperty ;
  rdfs:domain db:Diagnosis ; rdfs:range xsd:integer .

db:Diagnosis rdfs:subClassOf                  # required
  [ a owl:Restriction ;
    owl:onProperty db:primaryDiagnosis ;
    owl:minCardinality 1 ] .
```

... und die konkreten **Fakten (ABox)** – inklusive Alignment des enum-Werts auf einen NCIt-Term und RDF-star für die Herkunft der Aussage:

```
db:diag-TCGA-A1 a db:Diagnosis ;
  db:describesCase db:case-TCGA-A1 ;
  db:primaryDiagnosis ncit:Cxxxx ; # "Acinar cell carcinoma"
  db:ageAtDiagnosis 58 .

# RDF-star: Provenienz & Konfidenz an der Aussage
<< db:diag-TCGA-A1 db:primaryDiagnosis ncit:Cxxxx >>
  prov:wasDerivedFrom gdc:submission ;
  db:confidence 1.0 .
```

Kernidee: Der numerische Messwert (Genexpression) bleibt in der anndata-Matrix. Im Graphen steht nur die Bedeutung – welche Diagnose, welcher Fall, welcher NCIt-Term – plus, über RDF-star, woher die Aussage stammt und wie sicher sie ist.

5 Designentscheidungen & offene Punkte

- **enum: literal oder Alignment?** Werte als reine Strings ablegen (einfach) oder auf NCIt/DO/ICD-O abbilden (semantisch reich, aber Pflege einer Alignment-Tabelle nötig). Empfehlung: Alignment für die klinisch zentralen Felder, literal für den Rest.
- **Global-as-View vs. Local-as-View.** Wie wird das Mapping formalisiert – Sichten der Zielontologie über GDC (GaV, einfacher) oder GDC-Quellen als Sichten der Ontologie (LaV, flexibler bei Quellwechsel)? Für eine Quelle genügt zunächst GaV.
- **IRI-/Namespace-Strategie.** Stabile, wiederverwendbare IRIs für Klassen, Properties und Instanzen festlegen (z. B. TCGA-Barcode als Instanz-IRI).
- **Ontology-Evolution.** Das GDC-Schema ändert sich – das Mapping muss versioniert und bei Schemaänderungen nachgeführt werden (dein Flexibilitätsziel; Bezug zu ADR-0002).

- **Werkzeug für das Mapping.** Eigenes Python-Skript mit rdflib, oder ein deklaratives Mapping (z. B. RML/YARRRML)? Deklarativ ist wartbarer, rdflib flexibler.

6 Nächste Schritte

- **Übergabeformat mit dem API-Kollegen fixieren** (welche GDC-Nodes/Felder kommen wie an).
- **Zielontologie-Kern** aus dem Wissensnetz-Schema ableiten (Klassen/Properties, die das Mapping bedient).
- **Proof-of-Concept:** einen Node (z. B. diagnosis) end-to-end mappen, in Fuseki laden, per SPARQL abfragen.
- **Alignment-Tabelle** für die wichtigsten enum-Felder gegen NCIt/DO beginnen (via OLS/BioPortal).

7 Quellen

- [GDC Data Dictionary – Dokumentation](#)
- [NCI-GDC/gdcdictionary – YAML-Schemas \(GitHub\)](#)
- [W3C – RDF 1.2 / SPARQL 1.2 \(RDF-star\)](#)
- [EBI Ontology Lookup Service \(OLS\) – NCIt, DO nachschlagen](#)
- [RML / YARRRML – deklaratives Mapping nach RDF](#)
- docs/adr/0002-graph-db-wahl-offen.md – Graph-Modell-Entscheidung (RDF/OWL, RDF-star)
- wissensnetz/Wissensnetz_Konzept-Entwurf.docx – übergeordnetes Konzept